МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №1

Дисциплина: ТЕХНОЛОГИИ СОЗДАНИЯ И ПОДДЕРЖКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Тема: «Пользовательские истории и варианты использования»

Индивидуальное задание: сервис Flowra – персональный планировщик задач.

Работу выполнила: _____ Баннова.Д.А

Направление подготовки: 09.03.03 Прикладная информатика

Преподаватель: профессор кафедры анализа данных и искусственного интеллекта факультета компьютерных технологий и прикладной математики, к. техн. н., доцент

_____ Р.Ю. Вишняков

Краснодар

2026

Цель работы:

Изучить методики пользовательских историй и вариантов использования. Применить обе методики к своему проекту и описать, как пользователи будут взаимодействовать с системой.

Введение:

В качестве индивидуального задания для выполнения лабораторной работы была выбрана тема разработки интеллектуального приложения-планировщика Flowra. Разрабатываемая система представляет собой цифровой инструмент управления временем, сочетающий функции персонального планировщика, системы оптимизации задач и гибкой работы с временными блоками.

Основная цель приложения – автоматизация распределения задач пользователя с учётом его индивидуального режима дня, доступного времени, дедлайнов и уровня нагрузки. Система позволяет формировать адаптивное расписание на основе математических методов оптимизации, которое пользователь может редактировать вручную, при этом изменения динамически учитываются алгоритмом планирования.

Для описания требований к системе используются методики пользовательских историй и вариантов использования. Пользовательские истории позволяют формулировать требования на естественном языке, отражая реальные потребности пользователей, тогда как варианты использования обеспечивают более формализованное представление взаимодействия с системой. В рамках данной лабораторной работы описаны пользовательские истории системы в количестве 63 единиц и 15 детализированных вариантов использования.

Теоретическая часть:

1.1 Пользовательские истории (ПИ)

Пользовательские истории – способ описания требований к разрабатываемой системе, сформулированных как одно или более предложений на повседневном или деловом языке пользователя. Методика ПИ традиционно применяется в гибких методологиях разработки и предназначена для оперативного и экономического реагирования на изменяющиеся требования реального мира.

Для описания пользовательской истории используется следующий формат:

Как {роль пользователя}, я могу {делать что-нибудь}, для того чтобы {получить некоторую выгоду}

1.2 Преимущества и недостатки пользовательских историй

Преимущества: краткость и быстрота документирования; связность – позволяют обсуждать требования на протяжении всего проекта; неприхотливость в обслуживании; поддержка близкого контакта с клиентом; возможность разбить проект на небольшие этапы.

Недостатки: открытость для интерпретации при отсутствии приёмочных испытаний; затратность при необходимости тесного контакта с клиентом; плохая масштабируемость на больших проектах; отсутствие понятной конечной формализации.

1.3 Варианты использования (ВИ)

Варианты использования – это описание поведения системы, когда она взаимодействует с кем-то или чем-то из внешней среды. ВИ рассматривают систему как «чёрный ящик»: взаимодействия с системой, включая системные ответы, описываются точки зрения внешнего наблюдателя, при этом внимание сосредотачивается на том, что система должна сделать, а не как это должно быть сделано.

В данной работе применяется детализированный уровень описания ВИ, включающий разделы: Название, Цель, Акторы, Начально состояние, Основной сценарий, Альтернативные сценарии. При написании сценариев применяется правило чередования шагов «Актор – Система».

1. Пользовательские истории приложения Flowra

Раздел 1. Добавление и управление задачами

- Как пользователь, я могу создать задачу с названием, описанием, длительностью и дедлайном, чтобы система имела полную информацию для точного планирования.
- Как пользователь, я могу редактировать любой параметр уже созданной задачи, чтобы исправить ошибку или обновить информацию без удаления и пересоздания.
- Как пользователь, я могу удалить задачу из системы, чтобы расписание не содержало устаревших или отменённых задач.
- Как пользователь, я могу указать уровень приоритета задачи (1–5), чтобы система понимала, что делать в первую очередь при ограниченном времени.
- Как пользователь, я могу указать тип задачи — «глубокая работа» или «поверхностная», чтобы система размещала сложные задачи в часы максимальной концентрации.

– Как пользователь, я могу указать уровень когнитивной нагрузки задачи (1–5), чтобы алгоритм не ставил несколько трудных задач подряд.
– Как пользователь, я могу отметить задачу как выполненную, чтобы система автоматически обновила расписание и убрала задачу из активного плана.
– Как пользователь, я могу разрешить системе разбивать длинную задачу (4+ часов) на несколько коротких блоков, чтобы она помещалась в расписание.
– Как пользователь, я могу ввести задачу голосом через микрофон в свободном формате, чтобы быстро добавлять дела без набора текста.
– Как пользователь, я могу добавить задачу через ИИ-ассистента свободной фразой, чтобы система сама распознала параметры задачи из естественной речи.
– Как пользователь, я могу видеть список всех активных задач с их статусом, чтобы иметь полное представление о своей рабочей нагрузке.
– Как пользователь, я могу фильтровать и сортировать задачи по приоритету, типу и дедлайну, чтобы быстро находить нужное в большом списке.
– Как пользователь, я могу прикрепить задачу к фиксированному событию, чтобы видеть, что нужно подготовить к конкретной паре или встрече.
– Как пользователь, я могу задать подзадачи внутри основной, чтобы разбить крупный проект на шаги и отслеживать прогресс.
– Как пользователь, я могу получать предупреждение, если дедлайн невозможно выполнить при текущей нагрузке, чтобы не срывать сроки неожиданно.

Раздел 2. Профиль пользователя и персонализация

– Как пользователь, я могу пройти тест хронотипа при первом запуске, чтобы система с первого дня учитывала мои пики продуктивности.
– Как пользователь, я могу указать время сна и пробуждения, чтобы система никогда не ставила задачи на время отдыха.
– Как пользователь, я могу задать максимальное количество рабочих часов в день, чтобы система защищала меня от перегрузки.
– Как пользователь, я могу указать время поездки до учёбы, чтобы система учитывала время в пути как занятое.

– Как пользователь, я могу редактировать свою персональную кривую энергии, чтобы скорректировать алгоритм под свою реальность.
– Как пользователь, я могу добавить регулярные привычки, чтобы система резервировала время под них и не замещала их учебными задачами.
– Как пользователь, я могу настроить часовой пояс и язык интерфейса для комфортной работы из любой точки мира.
– Как пользователь, я могу сбросить настройки хронотипа и пройти тест заново, если мой режим изменился.

Раздел 3. Фиксированные события и расписание

– Как пользователь, я могу добавить расписание пар с поддержкой чётных и нечётных недель, чтобы знать свои «железные» занятости.
– Как пользователь, я могу добавить разовое фиксированное событие, чтобы система не назначала задачи на это время.
– Как пользователь, я могу редактировать и удалять фиксированные события, чтобы расписание всегда было актуальным.
– Как пользователь, я могу импортировать расписание из файла .ics (Google Календарь), чтобы не вводить десятки занятий вручную.
– Как пользователь, я могу видеть все события и задачи на одной временно́й шкале, чтобы иметь целостную картину дня.
– Как пользователь, я могу задать повторяющееся событие с гибкой периодичностью, чтобы не вносить его каждый раз вручную.

Раздел 4. Генерация и адаптация расписания

– Как пользователь, я могу нажать одну кнопку и получить оптимальный план на неделю, чтобы не тратить время на ручное планирование.
– Как пользователь, я могу видеть задачи в виде цветных временны́х блоков, чтобы мгновенно считывать структуру дня.
– Как пользователь, я могу перетаскивать временно́й блок задачи вручную, чтобы сохранить гибкость при изменениях.
– Как пользователь, я могу растягивать или сжимать блок для изменения длительности задачи прямо в интерфейсе.
– Как пользователь, я могу закрепить временно́й блок, чтобы алгоритм не перемещал его при авто-пересчёте.

– Как пользователь, я могу запустить пересчёт расписания одной кнопкой после правок, чтобы система адаптировалась под мои изменения.
– Как пользователь, я могу переключаться между видами дня, недели и месяца для разного горизонта планирования.
– Как пользователь, я могу видеть свободные «окна» в расписании, чтобы понимать, где есть резерв времени.
– Как пользователь, я могу видеть метрику загрузки дня (%), чтобы знать напряжённость дня заранее.
– Как пользователь, я могу отменять изменения в расписании (история версий), чтобы исправить неудачный пересчёт.
– Как пользователь, я могу получать предупреждение при загрузке дня более 85%, чтобы избежать выгорания.

Раздел 5. Адаптация и Rescue Mode

– Как пользователь, я могу нажать кнопку «Я не успел», чтобы система мгновенно перестроила план на остаток дня.
– Как пользователь, я могу активировать Rescue Mode при критическом перегрузе, чтобы оставить только самое важное.
– Как пользователь, я могу видеть, какие задачи система перенесла и куда, чтобы понимать изменения в плане.
– Как пользователь, я могу использовать Ctrl+Z для отмены ручного перемещения, чтобы быстро исправлять ошибки.
– Как пользователь, я могу выбрать «жёсткий» или «мягкий» режим планирования, чтобы настроить баланс между эффективностью и комфортом.

Раздел 6. ИИ-ассистент

– Как пользователь, я могу в свободной форме ставить задачи ИИ-модулю системы, чтобы добавлять их быстрее.
– Как пользователь, я могу корректировать параметры, предложенные системой, перед сохранением задачи.
– Как пользователь, я могу спрашивать систему «когда лучше делать сложные задачи», чтобы получать советы на основе моей энергии.
– Как пользователь, я могу попросить систему объяснить логику расписания, чтобы доверять ей.

– Как пользователь, я могу получать от системы предупреждение об «овербукинге» (избытке задач) до начала планирования.
– Как пользователь, я могу переключить голосовое управление, чтобы планировать «на ходу».

Раздел 7. Аналитика и прогресс

– Как пользователь, я могу видеть процент выполнения задач, чтобы объективно оценивать свою продуктивность.
– Как пользователь, я могу видеть график точности оценки времени (план vs факт), чтобы перестать недооценивать сложность дел.
– Как пользователь, я могу видеть, совпадает ли реальная работа с моей кривой энергии, чтобы корректировать хронотип.
– Как пользователь, я могу видеть тепловую карту нагрузки, чтобы выявлять закономерности своей усталости.
– Как пользователь, я могу получать еженедельный отчёт о продуктивности, чтобы иметь обратную связь о своей эффективности.
– Как пользователь, я могу видеть прогресс точности планирования, чтобы осознавать свой рост.

Раздел 8. Уведомления и мультиплатформенность

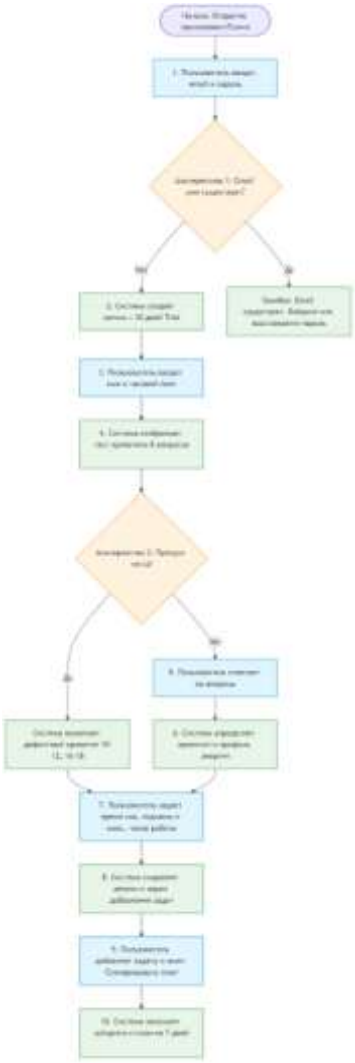
– Как пользователь, я могу получать напоминания за 15–30 минут до начала задачи, чтобы вовремя переключаться.
– Как пользователь, я могу получать утренний план дня, чтобы начинать утро с ясным фокусом.
– Как пользователь, я могу настроить предпочтительный канал уведомлений (Telegram, браузер, email).
– Как пользователь, я могу получать уведомление, если дедлайн скоро, а задача ещё не запланирована.
– Как пользователь, я могу настроить «режим тишины» на ночь, чтобы не получать уведомления в часы отдыха.
– Как пользователь, я могу работать в веб-версии на ПК и в Telegram Mini App на телефоне с мгновенной синхронизацией данных.

2. Варианты использования приложения Flowra

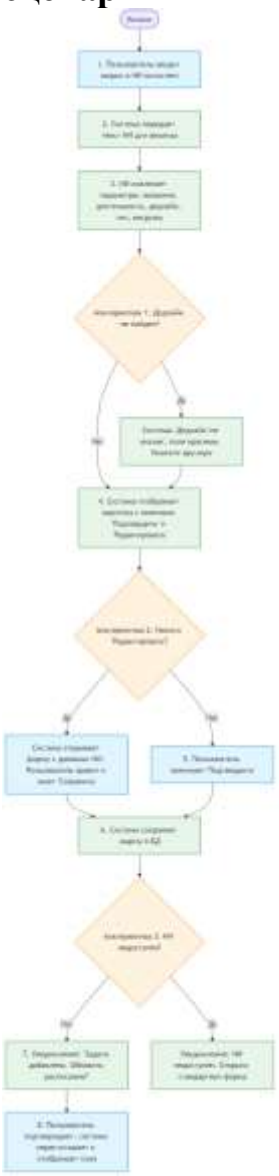
Ниже представлены 15 детализированных вариантов использования системы Flowra. Каждый ВИ оформлен по стандартному шаблону и включает

основной сценарий (описывающий типичный ход событий) и альтернативные сценарии (описывающие нетипичные и ошибочные ситуации). Сценарии написаны в формате чередования шагов «Пользователь — Система» без указания деталей интерфейса.
Для всех вариантов использования в данном разделе стейкхолдером является Пользователь как физическое лицо, заинтересованное в корректном функционировании системы планирования.

ВИ-01 «Зарегистрироваться и пройти онбординг»	
Цель	Создать персональный аккаунт и предоставить системе данные, необходимые для персонализированного планирования (хронотип, режим дня, расписание занятий).
Акторы	Пользователь
Начальное состояние	Пользователь открыл приложение Flowra впервые и не имеет учётной записи.

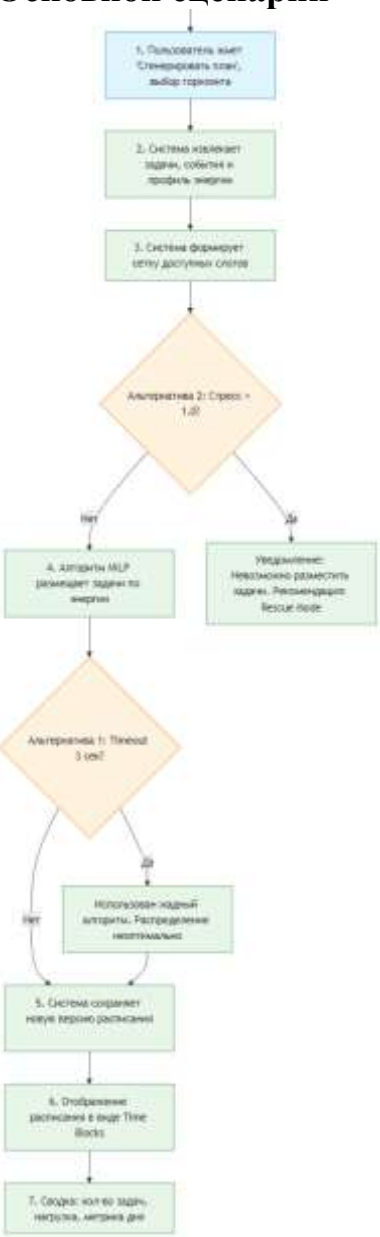
Основной сценарий 	**1.** Пользователь вводит адрес электронной почты и пароль и подтверждает регистрацию. **2.** Система создаёт учётную запись, назначает бесплатный пробный период (30 дней) и переходит к экрану онбординга. **3.** Пользователь вводит своё имя и часовой пояс. **4.** Система сохраняет данные и отображает тест хронотипа (8 вопросов о предпочтительном времени активности). **5.** Пользователь отвечает на вопросы теста. **6.** Система определяет хронотип (утренний/вечерний/промежуточный) и генерирует начальный профиль кривой энергии по часам. **7.** Пользователь задаёт время пробуждения, отхода ко сну и максимальное количество рабочих часов в день. **8.** Система сохраняет режим дня и отображает экран добавления первых задач. **9.** Пользователь добавляет не менее одной задачи и нажимает «Сгенерировать первый план». **10.** Система запускает алгоритм оптимизации и отображает готовое расписание на ближайшие 7 дней.
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 1 введён email, уже зарегистрированный в системе. **1.** Система сообщает об ошибке: «Пользователь с таким email уже существует. Войдите в аккаунт или восстановите пароль». Конец. **Альтернативный сценарий №2** **Предусловие:** На шаге 5 пользователь пропускает тест хронотипа. **1.** Система назначает промежуточный хронотип по умолчанию (пик продуктивности 10:00–12:00 и 16:00–18:00). **2.** Система информирует пользователя о том, что профиль можно уточнить позже в настройках. Продолжается с шага 7 основного сценария.

ВИ-02 «Добавить задачу через ИИ-ассистента»

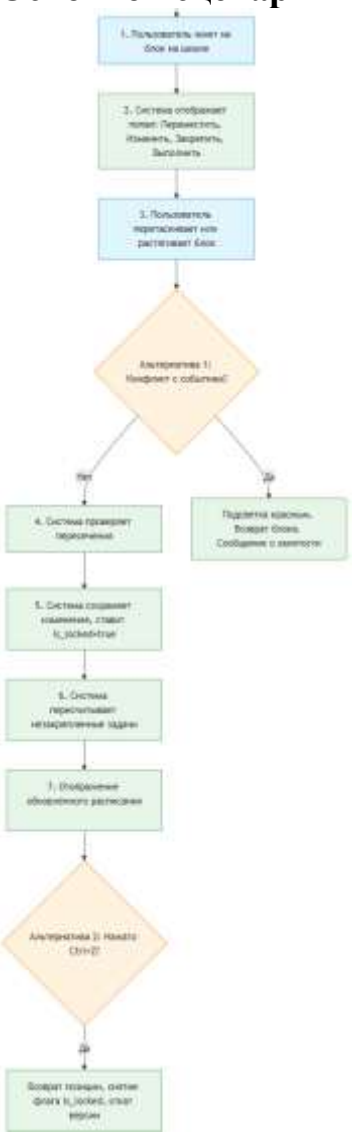
Цель	Создать новую задачу в системе, введя её описание в свободной форме на русском языке и получив автоматически заполненную карточку задачи.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован в системе.
Основной сценарий 	**1.** Пользователь открывает панель ИИ-ассистента и вводит текстовый запрос (например: «нужно написать реферат по истории, это сложно, сдать до пятницы, примерно 3 часа»). **2.** Система передаёт текст ИИ-ассистенту для анализа. **3.** ИИ-ассистент извлекает параметры задачи: название, длительность (180 мин), дедлайн (ближайшая пятница), тип (deep work), когнитивная нагрузка (4/5). **4.** Система отображает карточку задачи с предложенными параметрами и кнопками «Подтвердить» и «Редактировать». **5.** Пользователь проверяет параметры и нажимает «Подтвердить». **6.** Система сохраняет задачу в базе данных и добавляет её в очередь на планирование. **7.** Система уведомляет пользователя: «Задача добавлена. Обновить расписание?». **8.** Пользователь подтверждает обновление, система запускает пересчёт и отображает обновлённое расписание.
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 3 ИИ-ассистент не смог распознать дедлайн из текста. **1.** Система устанавливает дедлайн «не указан» и помечает поле красным. **2.** Система предлагает пользователю: «Не удалось определить дедлайн. Укажите вручную». **3.** Пользователь заполняет поле и продолжает с шага 5 основного сценария.

	Альтернативный сценарий №2 **Предусловие:** На шаге 4 пользователь нажимает «Редактировать». **1.** Система открывает полную форму задачи с предзаполненными полями от ИИ. **2.** Пользователь вносит изменения и нажимает «Сохранить». Продолжается с шага 6 основного сценария. **Альтернативный сценарий №3** **Предусловие:** ИИ-ассистент недоступен. **1.** Система уведомляет пользователя: «ИИ-ассистент временно недоступен. Добавьте задачу вручную». **2.** Система открывает стандартную форму создания задачи. Конец.

ВИ-03 «Сформировать план задач»	
Цель	Автоматически распределить все активные задачи пользователя по доступным временным слотам с учётом дедлайнов, приоритетов, хронотипа и фиксированных событий.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. В системе есть не менее одной активной задачи. Профиль пользователя (режим дня, кривая энергии) заполнен.

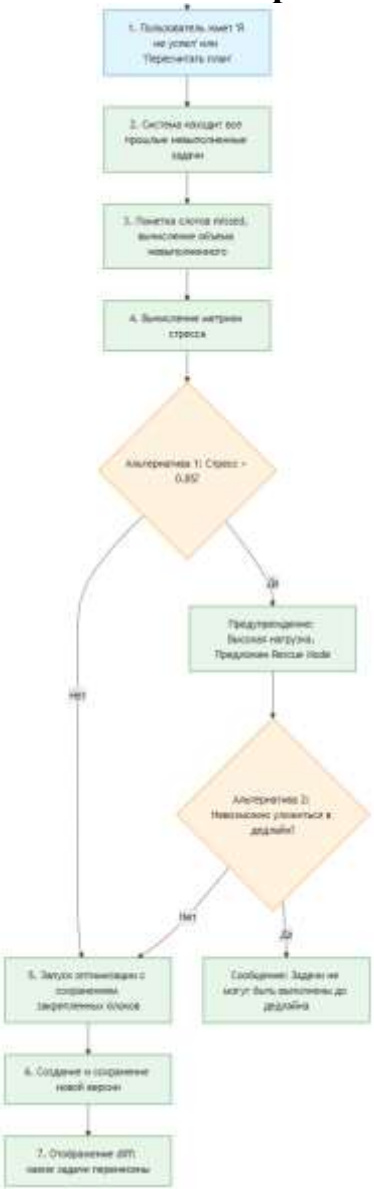
Основной сценарий 	**1.** Пользователь нажимает кнопку «Сгенерировать план» и выбирает горизонт планирования (1 день / 1 неделя). **2.** Система извлекает из базы данных все активные задачи, фиксированные события и профиль энергии пользователя. **3.** Система формирует временную сетку доступных слотов, исключая часы сна, фиксированные события и заблокированные блоки. **4.** Алгоритм оптимизации (MILP) решает задачу распределения: размещает сложные задачи в часы высокой энергии, минимизирует перегрузку. **5.** Система сохраняет результат в базе данных как новую версию расписания. **6.** Система отображает пользователю готовое расписание в виде цветных Time Blocks на временной шкале. **7.** Система показывает сводку: количество задач в плане, общая нагрузка в часах, метрика загрузки дня.
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** Алгоритм не нашёл оптимального решения за 3 секунды (превышен timeout). **1.** Система автоматически переключается на резервный жадный алгоритм (greedy). **2.** Система строит расписание и уведомляет пользователя: «Использован упрощённый алгоритм. Часть задач может быть распределена неоптимально». **3.** Продолжается с шага 5 основного сценария. **Альтернативный сценарий №2** **Предусловие:** Суммарный объём задач превышает доступное время (stress > 1.0). **1.** Система не запускает алгоритм.

	2. Система уведомляет пользователя: «Невозможно разместить все задачи. Доступно X часов, нужно Y часов. Рекомендуем использовать Rescue Mode или перенести задачи с меньшим приоритетом». **3.** Пользователь выбирает действие: активировать Rescue Mode или вручную отложить задачи. Конец.

ВИ-04 «Редактировать временной блок (Time Block)»	
Цель	Вручную изменить время или длительность запланированной задачи с автоматической проверкой конфликтов и обновлением расписания.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. Расписание сгенерировано и отображается на временной шкале.
Основной сценарий 	**1.** Пользователь нажимает на временной блок задачи на временной шкале. **2.** Система отображает попап с деталями задачи и кнопками «Переместить», «Изменить длительность», «Закрепить», «Отметить выполненной». **3.** Пользователь перетаскивает блок на новое время (или растягивает его границу для изменения длительности). **4.** Система проверяет: не пересекает ли новое положение фиксированные события, часы сна или другие задачи. **5.** Система сохраняет изменение и помечает блок как закреплённый (is_locked = true). **6.** Система автоматически пересчитывает расположение незакреплённых задач вокруг изменённого блока. **7.** Система отображает обновлённое расписание с подсветкой изменённого блока.

Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 4 новое положение блока конфликтует с фиксированным событием. **1.** Система подсвечивает недопустимую зону красным цветом в режиме реального времени при перетаскивании. **2.** При отпускании блока система возвращает его на исходную позицию и выводит сообщение: «Этот слот занят: [название события]». **3.** Пользователь выбирает другое время. Конец. **Альтернативный сценарий №2** **Предусловие:** Пользователь хочет отменить перемещение сразу после него. **1.** Пользователь нажимает Ctrl+Z (или кнопку «Отменить» в интерфейсе). **2.** Система возвращает блок на исходную позицию, снимает флаг is_locked и восстанавливает предыдущую версию расписания. Конец.

ВИ-05 «Пересчитать расписание (Reschedule)»	
Цель	Обновить расписание после того, как пользователь не выполнил одну или несколько задач, перераспределив их с учётом оставшегося свободного времени.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. Существует расписание с незавершёнными задачами, чьё запланированное время уже прошло.

Основной сценарий 	**1.** Пользователь нажимает кнопку «Я не успел» рядом с незавершённой задачей или кнопку «Пересчитать план». **2.** Система находит все задачи, чьи временные слоты находятся в прошлом и не помечены как выполненные. **3.** Система помечает эти слоты как missed и вычисляет общий объём невыполненных задач. **4.** Система вычисляет метрику стресса: stress = невыполненные_часы / оставшееся_свободное_время. **5.** Система запускает алгоритм оптимизации для перераспределения задач по оставшейся части дня/недели, сохраняя закреплённые блоки. **6.** Система создаёт новую версию расписания и сохраняет её в базе данных. **7.** Система отображает пользователю diff: что изменилось — какие задачи перенесены и куда.
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 4 метрика стресса превышает 0.85 (перегруз). **1.** Система выводит предупреждение: «Высокий уровень нагрузки. Рекомендуем Rescue Mode». **2.** Пользователь выбирает: продолжить обычный пересчёт или активировать Rescue Mode. **3.** Если выбран Rescue Mode — выполняется ВИ «Активировать Rescue Mode». Конец. **Альтернативный сценарий №2** **Предусловие:** Невозможно разместить все невыполненные задачи до их дедлайнов. **1.** Система сообщает: «Задачи [список] не могут быть выполнены до дедлайна. Рекомендуем отложить или уменьшить объём».

	2. Система показывает варианты действий для каждой конфликтующей задачи. Конец.

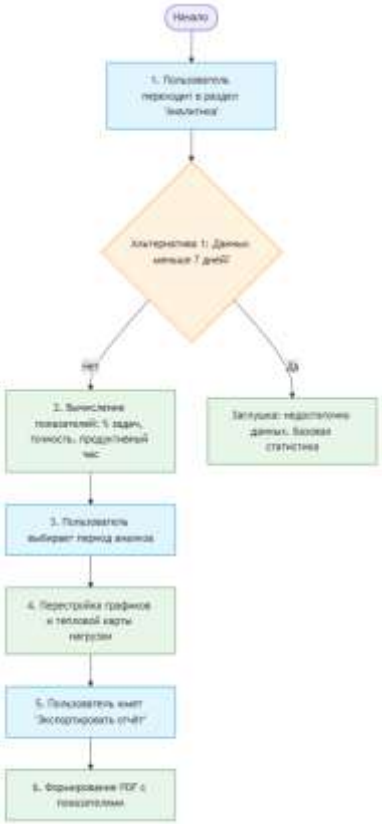
ВИ-06 «Настроить профиль и режим дня»	
Цель	Ввести или обновить персональные параметры (режим сна, расписание, хронотип), на основе которых строится персонализированное расписание.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. Открыт раздел «Настройки».
Основной сценарий	**1.** Пользователь выбирает раздел настроек профиля. **2.** Система отображает текущие параметры: время сна/пробуждения, максимум рабочих часов, хронотип, кривую энергии. **3.** Пользователь изменяет один или несколько параметров (например, сдвигает время пробуждения). **4.** Система в режиме реального времени обновляет превью кривой энергии на графике. **5.** Пользователь нажимает «Сохранить». **6.** Система сохраняет обновлённый профиль и инвалидирует кэш текущего расписания. **7.** Система предлагает: «Настройки изменились. Пересчитать расписание?». **8.** Пользователь подтверждает. Система запускает перегенерацию расписания и отображает результат.
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 5 пользователь нажимает «Отмена».

	1. Система откатывает все несохранённые изменения и возвращает прежние значения полей. Конец. **Альтернативный сценарий №2** **Предусловие:** Пользователь хочет пересдать тест хронотипа заново. **1.** Пользователь нажимает «Пройти тест хронотипа заново». **2.** Система отображает опросник из 8 вопросов. **3.** Пользователь отвечает на вопросы. **4.** Система пересчитывает хронотип и обновляет кривую энергии. **5.** Продолжается с шага 5 основного сценария.

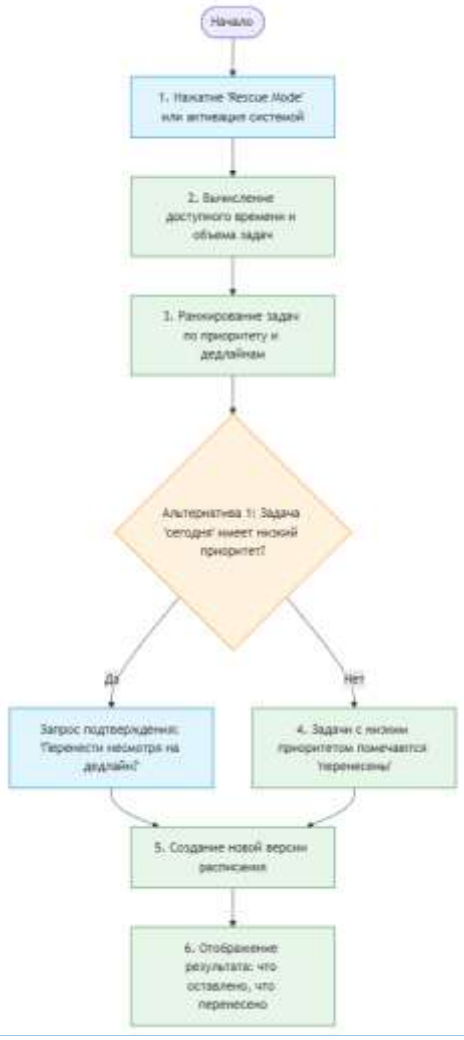
ВИ-07 «Добавить расписание занятий (фиксированные события)»	
Цель	Внести в систему учебное расписание пользователя, чтобы алгоритм блокировал эти временные слоты и не назначал на них задачи.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. Открыт раздел «Расписание занятий».
Основной сценарий	**1.** Пользователь выбирает способ добавления: вручную или импорт файла .ics. **2.** Система отображает форму добавления события (при ручном вводе) или диалог выбора файла (при импорте). **3.** Пользователь вводит данные события: название, день недели, время начала, время окончания, периодичность (еженедельно / чётная / нечётная неделя). **4.** Система валидирует данные: нет конфликтов с уже существующими событиями в тот же временной слот. **5.** Система сохраняет событие и помечает соответствующие временные слоты как недоступные. **6.** Система отображает обновлённую таблицу событий и предлагает пересчитать расписание.
Альтернативные сценарии	**Альтернативный сценарий №1**

	Предусловие: Пользователь выбирает импорт .ics файла. **1.** Система читает файл и извлекает все события. **2.** Система отображает список найденных событий с предложением подтвердить импорт. **3.** Пользователь подтверждает. Система сохраняет все события. Продолжается с шага 6 основного сценария. **Альтернативный сценарий №2** **Предусловие:** На шаге 4 обнаружен конфликт с существующим событием. **1.** Система выделяет конфликтующий слот и сообщает: «Это время уже занято: [название события]». **2.** Пользователь изменяет время или отменяет добавление. Конец.

ВИ-08 «Просмотреть аналитику продуктивности»	
Цель	Получить структурированную статистику о выполнении задач, точности планирования и динамике продуктивности за выбранный период.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. В системе накоплены данные минимум за 7 дней (task_history не пуст).
Основной сценарий	**1.** Пользователь переходит в раздел «Аналитика». **2.** Система вычисляет и отображает ключевые показатели за текущую неделю: процент выполненных задач, среднее отклонение фактического времени от запланированного, самый продуктивный час дня. **3.** Пользователь выбирает период анализа (неделя / месяц). **4.** Система перестраивает графики: тепловую карту нагрузки по дням, график точности планирования, кривую продуктивности vs кривую энергии. **5.** Пользователь нажимает «Экспортировать отчёт». **6.** Система формирует PDF-отчёт с показателями и графиками и предлагает его сохранить.



Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 1 данных менее 7 дней (новый пользователь). **1.** Система отображает заглушку: «Данных пока недостаточно для построения аналитики. Используйте систему ещё [N] дней». **2.** Система показывает только базовую статистику: число созданных задач, число выполненных задач. Конец.

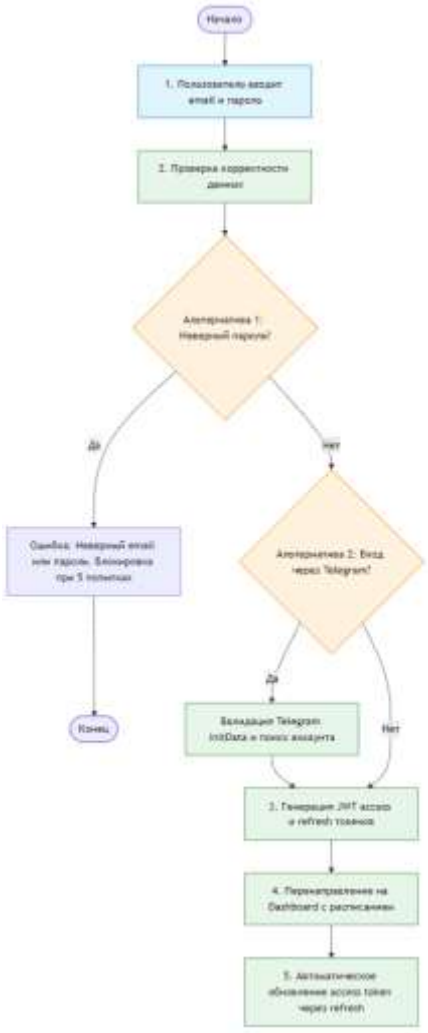
ВИ-09 «Активировать Rescue Mode»	
Цель	В условиях критического перегруза автоматически оставить в расписании только приоритетные задачи, безопасно перенеся остальные на будущие дни.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. Метрика стресса превышает 1.0 (невозможно выполнить все задачи в отведённое время).
Основной сценарий 	**1.** Пользователь нажимает кнопку «Rescue Mode» или система предлагает его активацию при обнаружении критического перегруза. **2.** Система вычисляет доступное время на оставшуюся часть дня и суммарный объём задач. **3.** Система ранжирует задачи по критичности: (приоритет × близость дедлайна) и оставляет в плане только задачи верхних приоритетов (P0 и P1), помещающиеся в доступное время. **4.** Задачи с меньшим приоритетом система помечает как «перенесены» и распределяет по ближайшим дням без создания цепного перегруза. **5.** Система создаёт новую версию расписания. **6.** Система отображает результат: «Оставлено X задач на сегодня. Перенесено Y задач. Ближайший перенос: завтра в [время]».

Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** Задача с дедлайном «сегодня» имеет низкий приоритет. **1.** Система НЕ переносит её автоматически. **2.** Система запрашивает явное подтверждение пользователя: «Задача [название] имеет дедлайн сегодня. Перенести несмотря на это?». **3.** Пользователь принимает решение вручную. Продолжается с шага 5 основного сценария.

ВИ-10 «Синхронизировать данные между платформами»	
Цель	Обеспечить актуальность задач и расписания при одновременном использовании веб-приложения и Telegram-бота.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. Telegram-аккаунт привязан к профилю Flowra.
Основной сценарий 	**1.** Пользователь добавляет задачу через Telegram-бот командой /add или свободным текстом. **2.** Telegram-бот передаёт данные в Backend API (POST /tasks). **3.** Система сохраняет задачу в базе данных и инвалидирует кэш расписания для данного пользователя. **4.** Пользователь открывает веб-приложение на компьютере. **5.** Веб-приложение запрашивает актуальные данные у API. **6.** Система возвращает обновлённый список задач, включая только что добавленную через бота. **7.** Пользователь видит актуальные данные без каких-либо ручных действий по синхронизации.

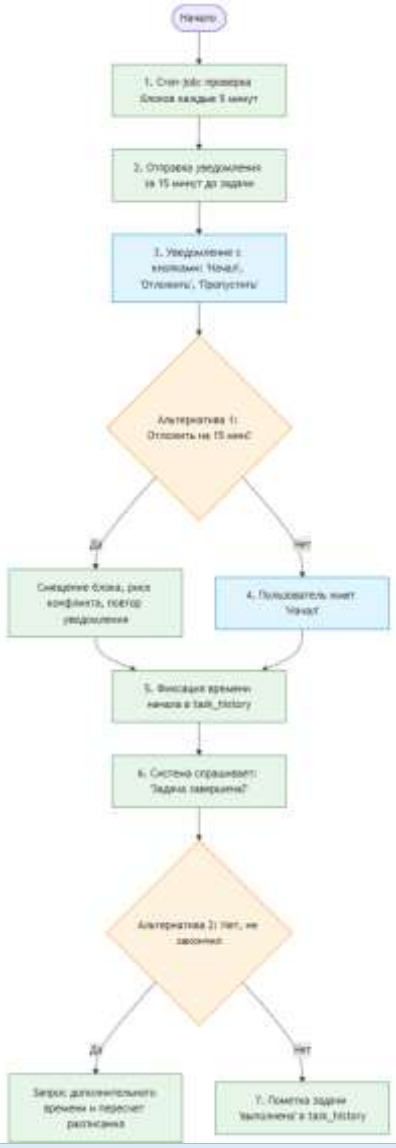
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 2 Backend API недоступен (сбой сети). **1.** Telegram-бот сохраняет задачу локально в буфер и уведомляет пользователя: «Задача сохранена локально и будет синхронизирована, когда соединение восстановится». **2.** При восстановлении соединения бот отправляет буферизованные данные. Продолжается с шага 3 основного сценария.

ВИ-11 «Войти в систему и авторизоваться»	
Цель	Аутентифицировать пользователя, выдать токен доступа и обеспечить безопасный доступ к персональным данным.
Акторы	Пользователь
Начальное состояние	У пользователя есть зарегистрированная учётная запись в системе Flowra.
Основной сценарий	**1.** Пользователь открывает страницу входа и вводит email и пароль. **2.** Система проверяет корректность введённых учётных данных. **3.** Система генерирует JWT access token (срок действия 15 минут) и refresh token (30 дней) и сохраняет refresh token в httpOnly cookie. **4.** Система перенаправляет пользователя на Dashboard с актуальным расписанием. **5.** При истечении access token браузер автоматически запрашивает новый, используя refresh token.



Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 2 введён неверный пароль. **1.** Система выводит сообщение «Неверный email или пароль». **2.** После 5 неудачных попыток система блокирует вход на 15 минут и предлагает восстановить пароль. Конец. **Альтернативный сценарий №2** **Предусловие:** Пользователь использует вход через Telegram. **1.** Система валидирует Telegram initData (проверяет HMAC-подпись). **2.** Система находит привязанный аккаунт по Telegram user_id. **3.** Продолжается с шага 3 основного сценария.

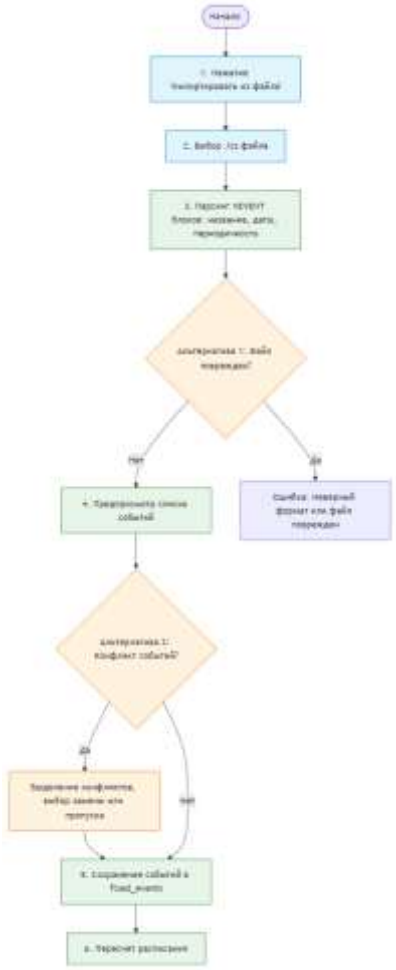
ВИ-12 «Получить уведомление и отреагировать на него»	
Цель	Своевременно получить напоминание о предстоящей задаче и быстро обновить её статус без необходимости открывать полный интерфейс приложения.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован, уведомления включены, Telegram-аккаунт привязан.

Основной сценарий 	**1.** Celery cron-job проверяет каждые 5 минут ближайшие временные блоки всех пользователей. **2.** За 15 минут до начала запланированной задачи система отправляет уведомление через выбранный канал (Telegram или браузер). **3.** Уведомление содержит: название задачи, время начала, длительность, кнопки «Начал», «Отложить на 15 мин», «Пропустить». **4.** Пользователь нажимает «Начал». **5.** Система фиксирует фактическое время начала в task_history. **6.** По истечении запланированной длительности система спрашивает: «Задача завершена?». **7.** Пользователь нажимает «Да». Система помечает задачу как выполненную и записывает фактическое время в task_history.
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 4 пользователь нажимает «Отложить на 15 мин». **1.** Система смещает временной блок на 15 минут и запускает повторное уведомление. **2.** Если откладываний было более 2 — система предупреждает: «Задача сдвинута. Возможен конфликт с [следующая задача]». Конец. **Альтернативный сценарий №2** **Предусловие:** На шаге 7 пользователь нажимает «Нет, не закончил». **1.** Система предлагает: «Сколько времени нужно дополнительно?». **2.** Пользователь указывает время. Система продлевает блок или создаёт новый слот для завершения задачи.

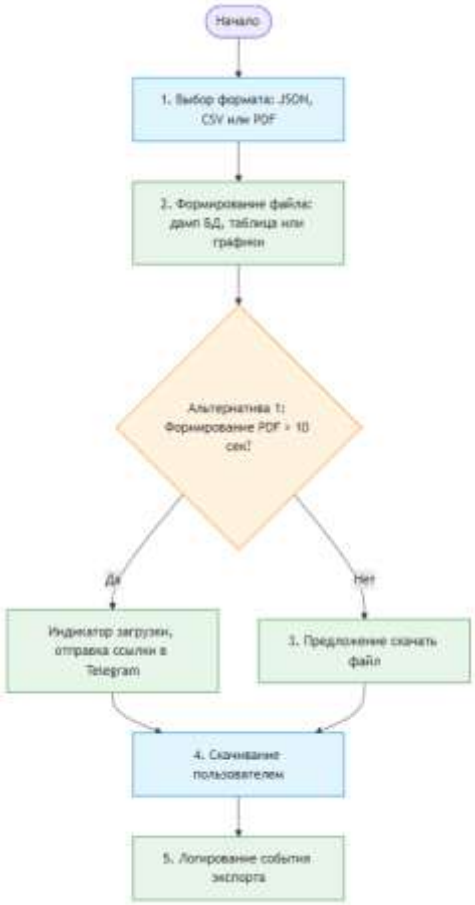
	3. Система пересчитывает расписание с учётом изменения. Конец.

ВИ-13 «Оформить подписку Pro»	
Цель	Приобрести платную подписку Pro для получения доступа к полному функционалу системы.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. Пользователь имеет бесплатный (Free) или истёкший тарифный план.
Основной сценарий	 **1.** Пользователь открывает страницу тарифов через баннер или раздел «Подписка». **2.** Система отображает сравнительную таблицу Free и Pro с детализацией отличий. **3.** Пользователь нажимает «Оформить Pro» и выбирает способ оплаты (ЮKassa или Telegram Stars). **4.** Система создаёт платёж и перенаправляет пользователя на страницу оплаты. **5.** Пользователь завершает оплату на стороне платёжной системы. **6.** Платёжная система отправляет webhook-уведомление об успешной оплате в Backend API. **7.** Система обновляет поля subscription_plan = 'pro' и subscription_expires_at = сегодня + 30 дней. **8.** Система отображает пользователю подтверждение: «Подписка Pro активирована до [дата]».
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** На шаге 5 пользователь отменил оплату или платёж отклонён. **1.** Платёжная система уведомляет Backend об отказе. **2.** Система оставляет текущий тарифный план без изменений.

	3. Система уведомляет пользователя: «Оплата не прошла. Попробуйте ещё раз или выберите другой способ оплаты». Конец.

ВИ-14 «Импортировать расписание из .ics файла»	
Цель	Автоматически перенести события из внешнего календаря (Google Календарь, iCal) в систему Flowra без ручного ввода.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. Пользователь имеет .ics файл с экспортированными событиями.
Основной сценарий 	**1.** Пользователь переходит в раздел «Расписание занятий» и нажимает «Импортировать из файла». **2.** Система открывает диалог выбора файла. Пользователь выбирает .ics файл. **3.** Система читает файл и парсит все события (VEVENT блоки): название, дата, время начала, время окончания, периодичность (RRULE). **4.** Система отображает найденные события в виде списка с предпросмотром для подтверждения. **5.** Пользователь просматривает список, при необходимости снимает галочки с ненужных событий, нажимает «Импортировать выбранные». **6.** Система сохраняет выбранные события в таблицу fixed_events и помечает соответствующие time_slots как недоступные. **7.** Система предлагает пересчитать расписание с учётом новых событий.
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** Файл повреждён или имеет неподдерживаемый формат. **1.** Система выводит ошибку: «Не удалось прочитать файл. Убедитесь, что файл имеет расширение .ics и не повреждён». **2.** Пользователь повторяет попытку с другим файлом. Конец. **Альтернативный сценарий №2**

	Предусловие: Одно из импортируемых событий конфликтует с уже существующим. **1.** Система выделяет конфликтующие события в списке предпросмотра. **2.** Пользователь решает: заменить существующее событие или пропустить импорт этого события. **3.** Система применяет выбранные действия. Продолжается с шага 6 основного сценария.

ВИ-15 «Экспортировать данные и сформировать отчёт»	
Цель	Сохранить все данные пользователя (задачи, расписание, аналитику) в виде файла для архивирования или анализа во внешних инструментах.
Акторы	Пользователь
Начальное состояние	Пользователь авторизован. В системе есть данные для экспорта.
Основной сценарий	 **1.** Пользователь открывает раздел «Настройки» → «Экспорт данных» или нажимает «Экспортировать отчёт» в разделе «Аналитика». **2.** Система предлагает выбор формата: JSON (полный дамп данных), CSV (таблица задач и статистики), PDF (форматированный аналитический отчёт). **3.** Пользователь выбирает формат и диапазон дат. **4.** Система формирует файл: для JSON — все задачи, события, расписание, task_history; для CSV — сводная таблица; для PDF — графики и ключевые показатели. **5.** Система предлагает сохранить или скачать файл. **6.** Пользователь скачивает файл. Система логирует событие экспорта.
Альтернативные сценарии	**Альтернативный сценарий №1** **Предусловие:** Формирование PDF занимает более 10 секунд (большой объём данных).

	1. Система выводит индикатор загрузки и сообщение «Формируем отчёт…». **2.** По готовности система отправляет ссылку на файл в Telegram или уведомление в браузере. Конец.

Ответы на контрольные вопросы

1**. Для чего используются варианты использования и пользовательские истории?**

Если говорить простыми словами, эти методики нужны для фиксации требований, но с разных сторон.

Пользовательские истории (User Stories) – это краткое описание потребностей на языке пользователя («хотелки»). Они нужны для общения с заказчиком и понимания ценности функции (зачем мы это делаем).

Варианты использования (Use Cases) – это детальные инструкции для разработчиков и тестировщиков. Они описывают поведение системы по шагам (алгоритм): что именно система должна сделать в ответ на действие пользователя. Они определяют границы системы и обрабатывают исключительные ситуации (ошибки).

2. **Какие обстоятельства затрудняют применение пользовательских историй?**

Главная сложность историй - их неформальность.

Зыбкость: Фраза «Хочу удобный поиск» может трактоваться по-разному. Без строгих критериев приемки разработчик может сделать не то, что нужно.

Масштаб: В огромных проектах историй становится слишком много (тысячи), в них легко запутаться и потерять целостность архитектуры.

Зависимость от диалога: Истории требуют постоянного контакта с заказчиком для уточнения деталей. Если заказчик недоступен, работа может встать.

3. **Какой уровень детализации традиционно применяется при описании ВИ?**

В профессиональной разработке под термином Use Case обычно понимают детализированный уровень. Это не просто заголовок, а полноценный документ, включающий название, акторов, предусловия, и, самое главное, пошаговый основной сценарий и альтернативные сценарии (обработку ошибок). Именно такой формат позволяет минимизировать риски при программировании.

Заключение

В ходе выполнения лабораторной работы были изучены и применены методики пользовательских историй и вариантов использования применительно к проекту Flowra – интеллектуальной системе планирования задач.

Составлено 63 пользовательских историй, охватывающих 8 функциональных групп системы. Разработаны 15 детализированных вариантов использования, каждый из которых содержит основной и альтернативные сценарии, сформулированные в соответствии с правилом чередования шагов «Актор – Система» без указания деталей реализации и интерфейса. Полученные документы составят основу для построения UML-диаграмм в рамках лабораторной работы №2.